

Nome:

André Filipe Caetano Mulati

Pedro Massaki Carnelossi

1. Python

Código original:

```
def adicionar(item, lista=[]):  
    lista.append(item)  
    return lista  
  
print(adicionar(1))  
print(adicionar(2))
```

Resultado da execução:



```
Python - execução original  
$ python ex1.py  
[1]  
[1, 2]
```

O que aconteceu: A saída foi [1] na primeira chamada e [1, 2] na segunda. No começo eu poderia achar que iria aparecer [2] na segunda chamada, mas isso não acontece porque a lista usada como valor padrão é reaproveitada.

Conceito da aula: O conceito aqui é o argumento padrão mutável. A lista criada em lista=[] fica guardada e é usada novamente quando a função é chamada outra vez. Para evitar isso, é melhor usar None e criar uma lista nova dentro da função.

```
def adicionar(item, lista=None):  
    if lista is None:  
        lista = []  
    lista.append(item)  
    return lista  
  
print(adicionar(1))  
print(adicionar(2))
```

Resultado depois da correção:



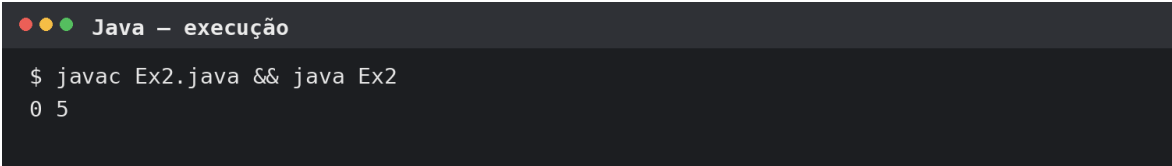
```
Python - depois da correção  
$ python ex1_corr.py  
[1]  
[2]
```

2. Java

Código original:

```
static void zera(int[] v, int n) {  
    v[0] = 0;  
    n = 0;  
}  
int[] v = {5, 5}; int n = 5;  
zera(v, n);  
System.out.println(v[0] + " " + n);
```

Resultado da execução:



```
Java - execução  
$ javac Ex2.java && java Ex2  
0 5
```

O que aconteceu: A saída é 0 5. O primeiro valor mudou porque o método alterou a posição 0 do mesmo array. Já o n continua 5 fora do método.

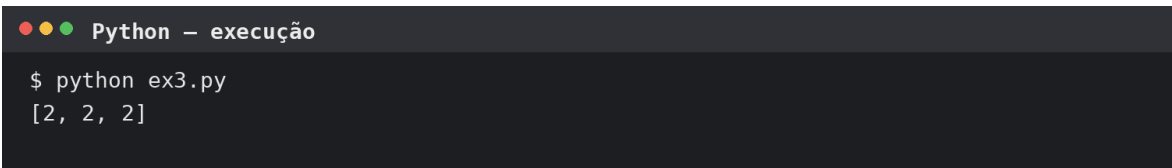
Conceito da aula: O conceito é passagem de parâmetros em Java. O Java passa os argumentos por valor. No caso do array, é passada uma referência para o mesmo objeto, então o conteúdo do array pode ser alterado. O int n é um valor primitivo, então a alteração feita dentro do método não muda a variável original.

3. Python

Código original:

```
fs = [lambda: i for i in range(3)]  
print([f() for f in fs])
```

Resultado da execução:



```
Python - execução  
$ python ex3.py  
[2, 2, 2]
```

O que aconteceu: A saída é [2, 2, 2]. No começo pode parecer que seria [0, 1, 2], mas as funções lambda ficam ligadas à variável i. Quando elas são executadas, o for já terminou e i está com o valor 2.

Conceito da aula: O conceito é closure e late binding. A função não guarda uma cópia do valor de i a cada volta do for; ela usa o valor da variável quando é chamada.

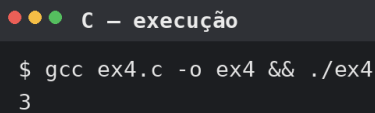
4. C

Código original:

```
int contador(void) {
    static int n = 0;
    return ++n;
}

// em main:
contador(); contador();
printf("%d\n", contador());
```

Resultado da execução:



```
C - execução
$ gcc ex4.c -o ex4 && ./ex4
3
```

O que aconteceu: A saída é 3. A primeira chamada deixa n igual a 1, a segunda deixa n igual a 2 e a terceira deixa n igual a 3. É esse terceiro valor que aparece no printf.

Conceito da aula: O conceito é a variável static. Ela é inicializada uma vez e não perde o valor quando a função termina. Por isso, nas próximas chamadas ela continua de onde parou.

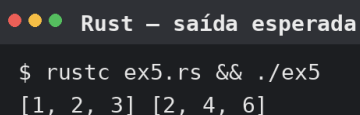
5. Rust

Código original:

```
fn dobra(v: Vec<i32>) -> Vec<i32> {
    v.iter().map(|x| x * 2).collect()
}

let v = vec![1, 2, 3];
let d = dobra(v);
println!("{:?} {:?}", v, d);
```

Resultado da execução:



```
Rust - saída esperada
$ rustc ex5.rs && ./ex5
[1, 2, 3] [2, 4, 6]
```

O que aconteceu: Nesse caso existe um erro de compilação no código original. O vetor v é passado para a função dobra e a posse dele é movida para a função. Depois disso, o programa tenta usar v novamente no println!, o que não é permitido.

Conceito da aula: O conceito é ownership (posse) do Rust. Quando v é passado para dobra, a função recebe a posse do Vec. Por isso, a variável original não pode ser usada novamente depois do move. Para corrigir, fiz uma cópia do vetor antes de passar v para a função.

Correção que fiz:

```
fn dobra(v: Vec<i32>) -> Vec<i32> {
    v.iter().map(|x| x * 2).collect()
}

let v = vec![1, 2, 3];
let original = v.clone();
let d = dobra(v);
println!("{:?} {:?}", original, d);
```

Resultado depois da correção: [1, 2, 3] [2, 4, 6]

6. Python

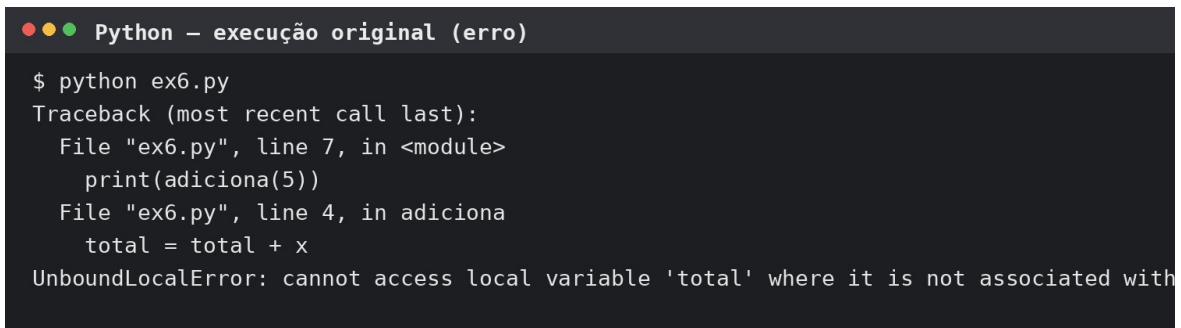
Código original:

```
total = 0

def adiciona(x):
    total = total + x
    return total

print(adiciona(5))
```

Resultado da execução:



```
Python - execução original (erro)
$ python ex6.py
Traceback (most recent call last):
  File "ex6.py", line 7, in <module>
    print(adiciona(5))
  File "ex6.py", line 4, in adiciona
    total = total + x
UnboundLocalError: cannot access local variable 'total' where it is not associated with
```

O que aconteceu: Aqui apareceu um erro: UnboundLocalError. Dentro da função existe uma atribuição para total, então o Python entende que total é uma variável local. O problema é que ela está sendo usada antes de receber um valor local.

Conceito da aula: O conceito é escopo de variáveis. Como eu queria usar a variável total que foi criada fora da função, coloquei global total dentro da função. Assim o Python passa a usar a variável global.

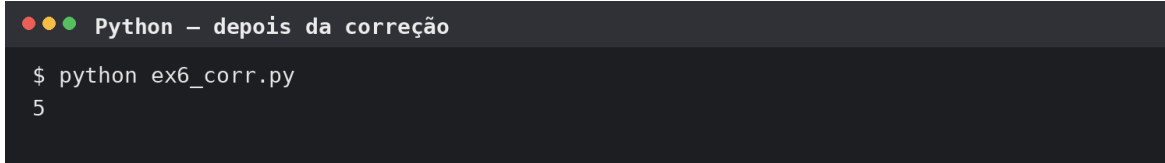
Correção que fiz:

```
total = 0

def adiciona(x):
    global total
    total = total + x
    return total

print(adiciona(5))
```

Resultado depois da correção:

A terminal window with a dark background. The title bar at the top shows three colored circles (red, yellow, green) followed by the text "Python – depois da correção". The terminal content shows a prompt "\$" followed by the command "python ex6_corr.py" on the first line, and the output "5" on the second line.

```
Python – depois da correção
$ python ex6_corr.py
5
```

Conclusão

Depois de executar os exemplos, deu para perceber que nem sempre um resultado diferente do que eu esperava significa que existe um erro no código. Alguns exemplos funcionam normalmente, mas o resultado depende de uma regra específica da linguagem. Os principais conceitos que apareceram foram argumentos padrão mutáveis, passagem de parâmetros, closure, static, ownership e escopo de variáveis.